Arquiteturas NeRF em Condições de Poucas Vistas: Uma Metodologia Sistemática de Benchmarking



unesp

Programa de Pós-Graduação em Ciência da Computação (PPGCC-UNESP)
Aprendizado Profundo - Prof. Dr. Denis Henrique Pinheiro Salvadeo

Nome: Felipe Gomes da Silva

Agenda

1. Introdução, Motivações e Justificativa	3
2. Fundamentação Teórica	4
3. Metodologia	6
4. Resultados e Discussão	12
5. Conclusão	15
Referências	16



Introdução, Motivações e Justificativa

NeRFs são o estado da arte em síntese de novas vistas.

- Custosos: exige muito poder computacional (VRAM, tempo de treino)
- Dependência de dezenas a centenas de vistas por cena

Cenários reais frequentemente violam essas premissas:

- Escaneamento mobile e histologia
- Datasets pequenos e câmeras esparsas por natureza

Não há nenhum benchmark sistemático e padronizado de modelos generalizáveis, assim como seu uso é um tanto quanto restrito.

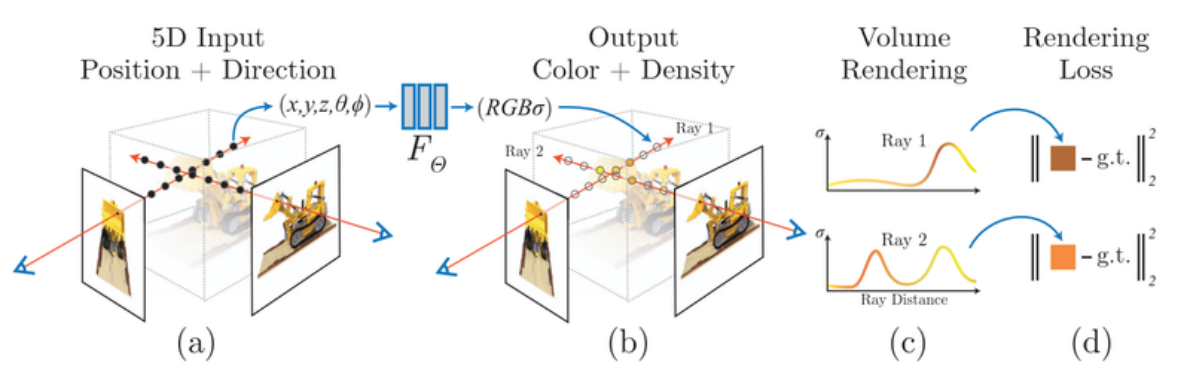
Para isso, foi feita uma análise comparativa extensa entre os modelos NeRF aqui dispostos, assim como a integração

Fundamentação Teórica - O que é um NeRF?

Modelo que aproxima uma função contínua 5D, com 2 componentes:

$$F_\theta : F_{\theta\prime} \circ \gamma(\mathbf{x}, d) \rightarrow (\sigma, \mathbf{c}), \quad \sigma \in \mathbb{R}_{\geq 0}, \; c \in [0,1]^3$$

- Extração de características: via γ (extrator de features)
- Rede $F_{\theta\prime}$ MLP original
- Renderização por integração volumétrica ao longo de raios



Fundamentação Teórica - Fine-Tuning

- Estratégias de Fine Tuning utilizadas

  A Loss

$$\mathcal{L}_{\text{photo}} = \frac{1}{|\mathcal{R}|} \sum_{\mathbf{r} \in \mathcal{R}} \|\hat{C}(\mathbf{r}) - C(\mathbf{r})\|_2^2$$

Full Fine-Tuning (Full-FT)

Os parametros da rede $\theta = \{W, b\}$ são todos atualizados simultaneamente: seja no backbone de feature 2D ou a rede de renderização 3D.

Partial Fine-Tuning:

Os updates se restringem a (separadamente): **Decoder (PFT-Decoder) e Backbone (PFT-Backbone)**

Selective PEFT: BitFit

Congela todos os pesos $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ e otimiza apenas os vetores de viés aditivo seguindo: $y = Wx + (b + \Delta b)$

Additive PEFT: Low-Rank Adaptation (PEFT-LoRA)

LoRA congela os pesos pré-treinados $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ e representa os updates de peso via uma decomposição low-rank:

$$W' = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA$$

onde $B \in \mathbb{R}^{d_{\text{out}} \times r}$ e $A \in \mathbb{R}^{r \times d_{\text{in}}}$ tem o rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$ escalado por α.

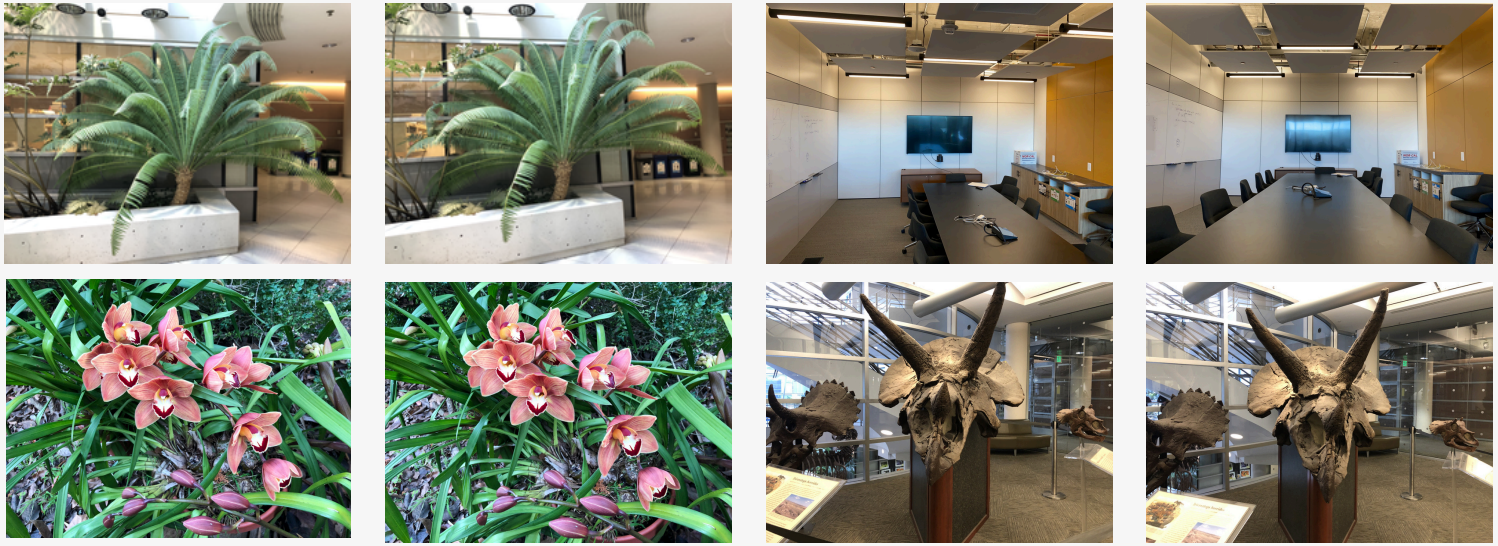
Metodologia

1. Aquisição do Dataset
2. Geração dos splits (uniform e random, semente fixa)
3. Treino / adaptação por modelo e regime
4. Avaliação no conjunto de teste
5. Coleta de métricas (PSNR, SSIM, LPIPS, FPS, VRAM)



Aquisição do Dataset

LLFF (Local Light Field Fusion) - [mildenhall et al. 2019]



- 20 diferentes vistas em 8 cenas frontais distintas



Estratégias de Otimização

Regimes e Modelos

Per Scene	FEW-SHOT / GENERALIZÁVEL
Otimiza estrutura de dados para cada cena isoladamente	Aprende priors em grandes datasets; generaliza para novas cenas com poucas vistas
InstantNGP → hash encoding multiescala	PixelNeRF → ResNet + projeção por pixel
Nerfacto → heurísticas combinadas (com hashing também)	GNT → Transformer em extrator e renderizador
Splatfacto → 3D Gaussian Splatting ao invés do ray marching	-
✓ Alta qualidade c/ dados ✗ Colapsa (supostamente) com vistas esparsas	✓ Robusto à escassez de vistas ✗ Menor fidelidade de detalhe

Aquisição do Dataset

- LLFFHOLD = 8 (teste = cada 8 imagens, 1 é teste) [Nyemeyer et al. 2023]
- downscale 8 (4k → 480 pixels)
- Regimes de Avaliação:

Zero Shot	Fine Tuning	Per-Scene
Weights only	Fine Tuning com (3, 6, 10) vistas	100% das vistas (70% para treino) ou as 3, 6, 10 vistas
PixelNeRF, GNT	PixelNeRF, GNT	Nerfacto e Instant-NGP

2 Estratégias de *sampling* das vistas:

- Uniform — interpolação linear sobre os índices da câmera (np.linspace)
- Random — sorteio com semente fixa (42 - reprodutibilidade)

Hardware: NVIDIA GTX 1660 Super · 6 GB VRAM · Tempo total de execução: ~180 horas

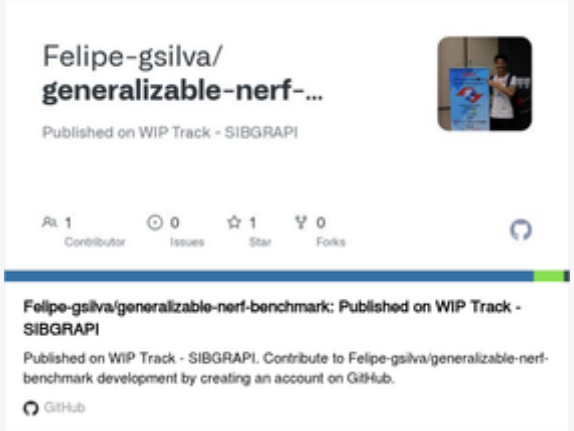
Implementação NeRFStudio

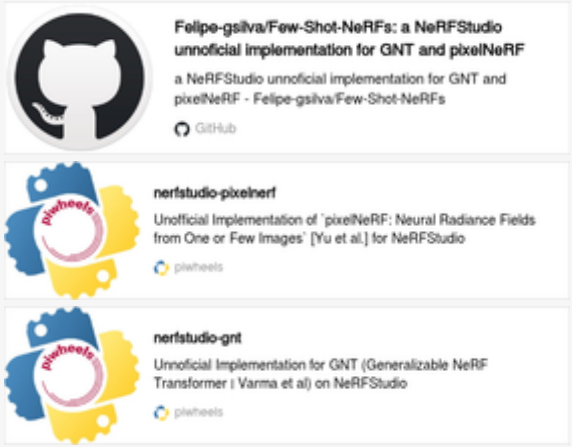
Framework unificado → elimina variáveis de implementação
Modelos:

- Nativos: Instant-NGP, Nerfacto e Splatfacto
- Portados: GNT e PixelNerf

Infraestrutura compartilhada:

- Otimizador: Adam η = 1e-3, weight decay = 0





Métricas

Qualidade Visual	Eficiência Computacional
PSNR ↑ Fidelidade pixel a pixel (dB)	Tempo de Convergência
SSIM ↑ (Índice de Similaridade Estrutural Luminância + contraste + estrutura)	Pico de VRAM (GB - Alocação durante treino)
LPIPS ↓ (Distância perceptual Backbone: **VGG**)	Tamanho do Modelo (parametros X precisão)
–	Throughput computacional (FPS)

Resultados e Discussão

Tabela 1: Resultados do benchmark no dataset LLFF. Valores representam as médias globais sobre as cenas de teste. Tempo de treino corresponde à configuração de 500steps no fine-tuning (melhor desempenho de ablation) para modelos generalizáveis e convergência de 30,000 steps para os modelos per-scene (N). VRAM indica o maximo de memoria dinamica alocada durante a inferencia.

Model	Regime	PSNR ↑	SSIM ↑	LPIPS ↓	Time ↓	FPS ↑	VRAM ↓
PixelNeRF	ZS	6.35	0.085	0.800	∼22 s	0.004	1.93
	FT	10.92	0.162	0.772	4 min 36 s	0.006	1.97
GNT	ZS	10.51	0.314	0.742	**∼9 s**	0.014	1.94
	FT	15.07	0.368	0.679	4 min 59 s	0.015	1.93
Instant-NGP	N=10	14.06	0.442	0.498	31 min 51 s	0.199	1.09
Nerfacto	N=10	**27.11**	**0.875**	**0.171**	8 min 57 s	0.388	1.43
Splatfacto	N=10	24.80	0.830	0.210	16 min 23 s	**200.7**	**0.10**

Tabela 2: Upper Bound NerfBaselines

Modelo	**PSNR (dB) ↑**	**SSIM ↑**	**LPIPS (VGG) ↓**	**Time**	**GPU mem. (GB)**
Mip-NeRF 360	26.59	0.846	0.168	9h 55m 23s	104.98
TensoRF	26.68	0.834	0.202	31m 32s	20.38

Fonte: Kulhanek et al, 2024

Resultados e Discussão

Tabela 3: Resultados da avaliação *multi-seed* das estratégias de *fine-tuning* no PixelNeRF (5 valid scenes) e no GNT (6 valid scenes) usando 3 seeds $\{42, 123, 999\}$ e 500 passos. PixelNeRF teve seu pico em PSNR sob a adaptação *PFT-Backbone*, enquanto o GNT performou melhor sob o tuning do decoder epipolar (*PFT-Decoder*) e *PEFT-LoRA*.

Model	Strategy	η	r	PSNR (dB) ↑	SSIM ↑	LPIPS ↓
PixelNeRF	Zero-Shot (ZS)	N/A	N/A	6.35 ± 0.05	0.085	0.800
	Full-FT	10^{-4}	N/A	9.03 ± 0.11	0.136	0.752
	PFT-Decoder	10^{-4}	N/A	10.58 ± 0.08	**0.205**	0.773
	PFT-Backbone	10^{-4}	N/A	**10.92** ± 0.26	0.162	0.772
	BitFit	10^{-4}	N/A	10.79 ± 0.19	0.119	0.775
	PEFT-LoRA	10^{-3}	4	8.59 ± 0.10	0.175	**0.725**
GNT	Zero-Shot (ZS)	N/A	N/A	10.51 ± 0.11	0.314	0.742
	Full-FT	10^{-3}	N/A	15.06 ± 0.03	**0.369**	0.681
	PFT-Decoder	10^{-3}	N/A	**15.07** ± 0.03	0.368	**0.679**
	PFT-Backbone	10^{-4}	N/A	10.51 ± 0.11	0.314	0.742
	BitFit	10^{-3}	N/A	13.12 ± 0.05	0.358	0.742
	PEFT-LoRA	10^{-3}	4	14.57 ± 0.07	**0.369**	0.706

Resultados e Discussão

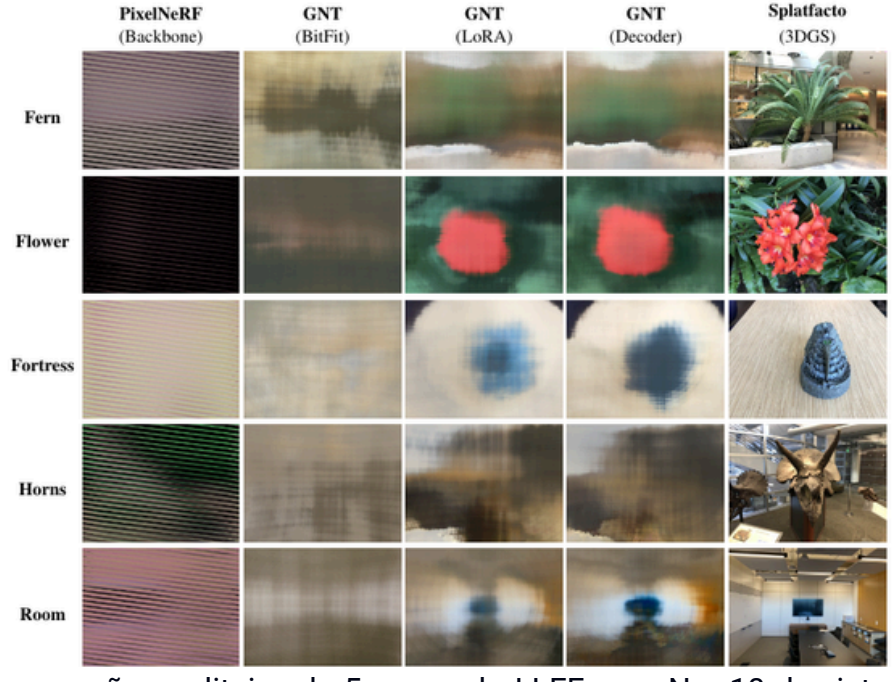


Figura 1: Comparação qualitaiva de 5 cenas do LLFF com N = 10 de vistas de entrada. Da esquerda para a direita: PixelNeRF usou PFT-Backbone (Col. 1); GNT usou Bitfit (Col. 2), PEFT-LoRA (r = 4, Col. 3) e PFT-Decoder (Col. 4); Splatfacto sem FT (Col. 5). Aumentar o numero de parametros adaptados no GNT aumentou os detalhes de alta frequência e contornos de objetos, enquanto o Splatfacto preservou os detalhes com throughput > 200 FPS.

Conclusão

- Vemos que o *Transfer Learning* consegue melhorar o modelo mesmo com poucas vistas.
- Os generalizáveis apresentaram dificuldades devido à inércia arquitetural.
- **Computacionalmente**: mais rápido adaptar os generalizáveis, porém em inferência, os per-scene são melhores.
- **Em inferência:** o Splatfacto domina (> 200 FPS)
- **Em qualidade visual**: o Nerfacto ainda predomina, apesar do Splatfacto ficar próximo

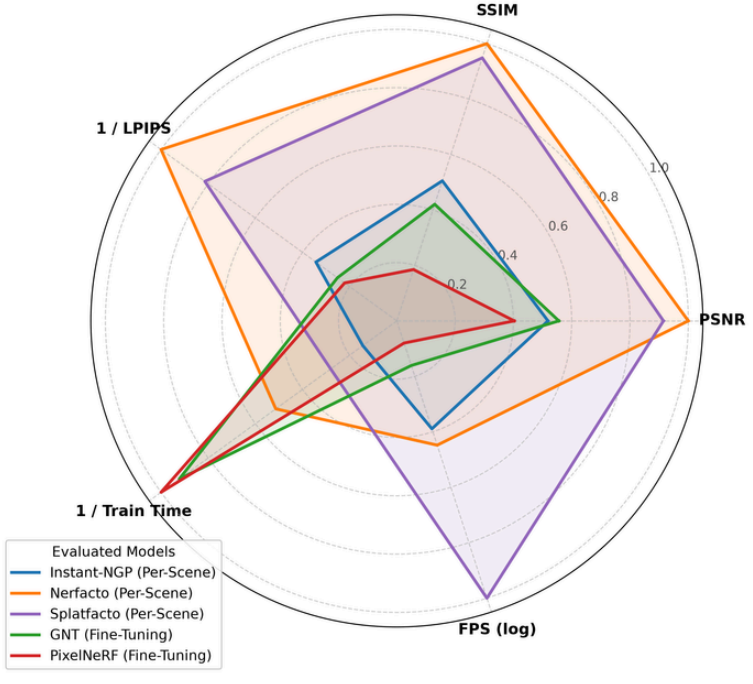


Figura 2: Trade-offs de várias métricas sob condições de vistas esparsas ($N = 10$). Métricas foram normalizadas entre $[0, 1]$ (perímetro distal = mais desempenho), usando $\frac{1}{\text{LPIPS}}$, $\frac{1}{\text{Train Time}}$, and FPS escalado por log.

REFERÊNCIAS

- Tancik et al. **Nerfstudio: A Modular Framework for Neural Radiance Field Development.** 2023
- Yu et al. **pixelNeRF: Neural Radiance Fields from One or Few Images**. 2021
- Wang et al. **Is Attention All That NeRF Needs?.** 2023
- Hedman et al. **Baking Neural Radiance Fields for Real-Time View Synthesis.** 2021
- Niemeyer et at. **RegNeRF: Regularizing Neural Radiance Fields for View Synthesis from Sparse Inputs.** 2021
- Kulhanek et al. **NerfBaselines: Consistent and Reproducible Evaluation of Novel View Synthesis Methods**. 2024
- Wu et al. **ReconFusion: 3D Reconstruction with Diffusion Priors.** 2024.
- Seo et al. **Improving Sparse-View 3DGS Generalization via Flat Minima Optimization.** 2026
- Gupta et al. **Beyond {LoRA}: Exploring Efficient Fine-Tuning Techniques for Time Series Foundational Models.** 2024



Muito obrigado!

felipe.gomes-silva@unesp.br

unesp